

Universidad de San Carlos de Guatemala

Division de las Ciencias de la Ingenieria

Ingenieria en Sistemas

Organizacion De Lenguajes Y Compiladores 1

Seccion: A

Auxiliar: Carlos Fernando Enrique Lopez Garcia



## **“Manual de Usuario Proyecto Golite Fase 1”**

**Estudiante:**

Mynor Miguel Monzón Martínez - 3195097021306

Guatemala, 12 de junio de

2026

# Introduccion

Este manual describe el uso del Interprete GoLite, una aplicacion de escritorio desarrollada en Java que permite escribir, editar y ejecutar codigo en el lenguaje GoLite directamente desde una interfaz grafica. GoLite es un subconjunto del lenguaje Go disenado con fines educativos para el curso de Organizacion de Lenguajes y Compiladores 1.

El sistema permite abrir y guardar archivos de codigo con extension .glt, ejecutar el interprete sobre el codigo escrito, visualizar la salida en una consola integrada, y consultar el reporte de tokens reconocidos y los errores detectados durante el analisis.

## 1. Requisitos del Sistema

Para ejecutar el Interprete GoLite se necesita lo siguiente:

- Sistema operativo: Ubuntu Linux (recomendado), o cualquier sistema con soporte para Java.
- Java Runtime Environment (JRE) version 17 o superior.
- No se requiere instalacion adicional de software. El programa se distribuye como un archivo JAR ejecutable que incluye todas sus dependencias.

## 2. Instalacion y Ejecucion

### 2.1 Obtener el programa

El programa se distribuye como un archivo JAR ejecutable. Se puede obtener de dos formas:

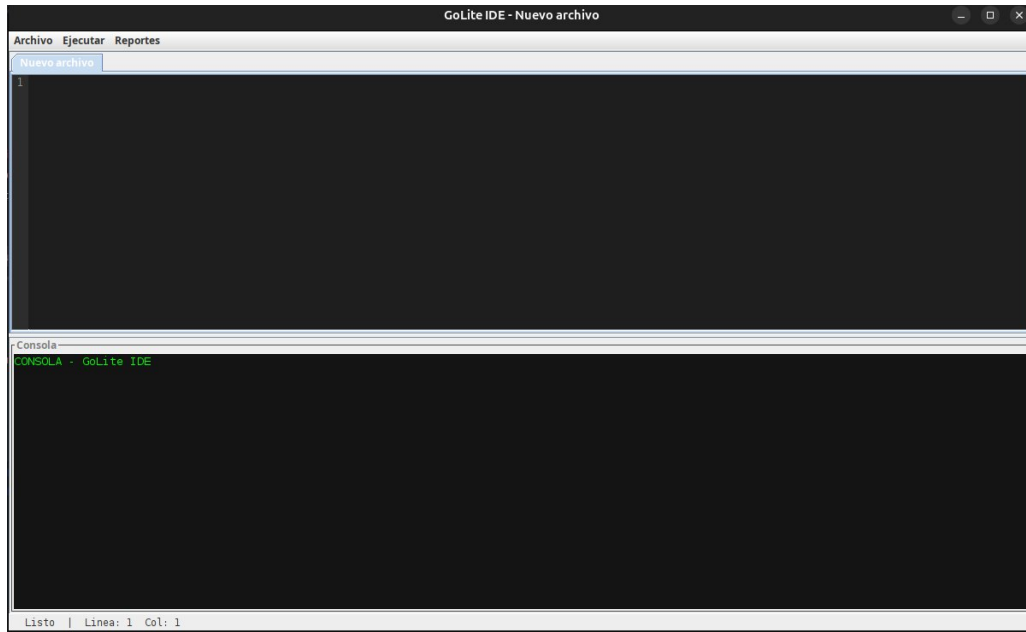
- Descargando el Release publicado en el repositorio de GitHub del proyecto.
- Compilando el codigo fuente con Maven desde la carpeta GoLiteInterpreter/ del repositorio.

### 2.2 Ejecutar el programa

Para iniciar el interprete, abrir una terminal en la carpeta donde se encuentra el archivo JAR y ejecutar el siguiente comando:

```
java -jar GoLiteInterpreter-1.0-SNAPSHOT.jar
```

Al ejecutarlo, la ventana principal del interprete se abrira automaticamente.



### 3. Interfaz Principal

La ventana principal del interprete esta dividida en las siguientes areas:

| Area               | Descripcion                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| Editor de codigo   | Area principal donde se escribe o pega el codigo GoLite. Muestra la linea actual del cursor.                |
| Consola de salida  | Panel inferior donde aparece la salida generada por <code>fmt.Println</code> y los mensajes del interprete. |
| Barra de menu      | Contiene los menus Archivo (nuevo, abrir, guardar) y Ejecutar.                                              |
| Tabla de tokens    | Pestana o ventana que muestra todos los tokens reconocidos durante el analisis lexico.                      |
| Reporte de errores | Pestana o ventana que muestra todos los errores detectados con su tipo, linea y columna.                    |

## 4. Funcionalidades del Sistema

### 4.1 Crear un archivo nuevo

Para limpiar el editor y comenzar con un archivo en blanco, ir al menu Archivo y seleccionar Nuevo. Si hay cambios sin guardar, el editor se limpiara de inmediato.

### 4.2 Abrir un archivo .glt

Para abrir un archivo de codigo GoLite existente, ir al menu Archivo y seleccionar Abrir. Se abrira un explorador de archivos donde se debe navegar hasta el archivo con extension .glt y seleccionarlo. El contenido del archivo se cargara en el editor automaticamente.

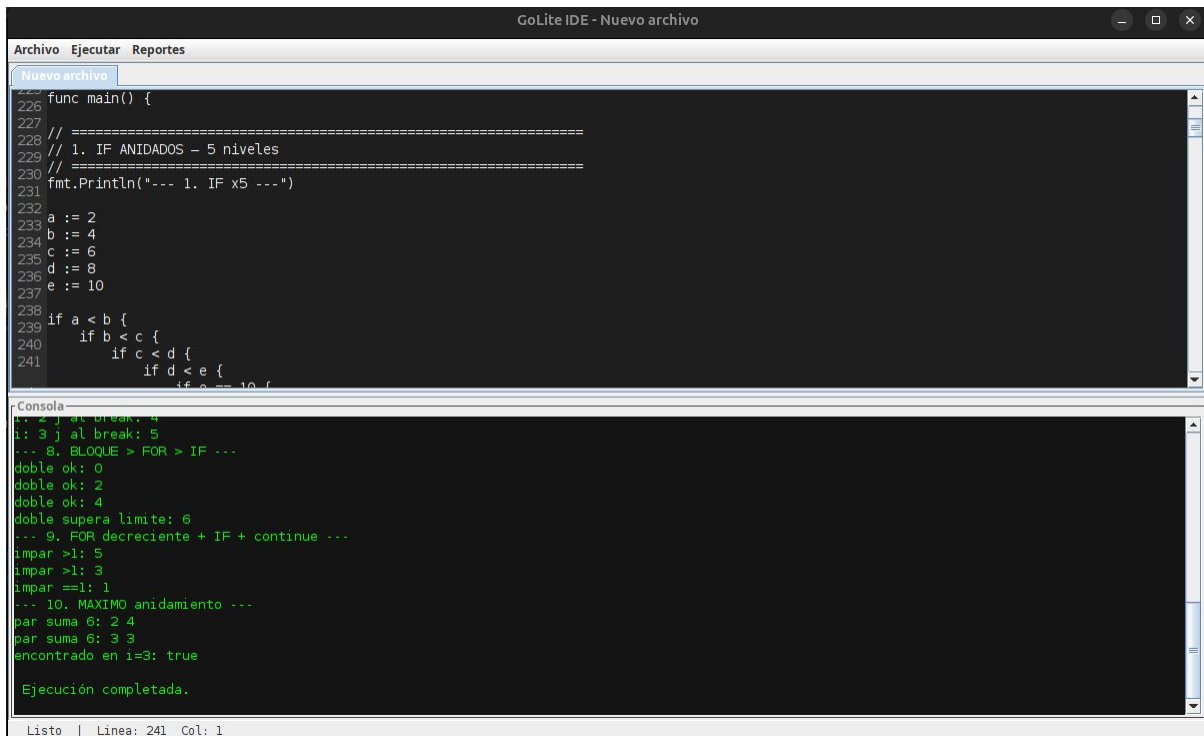
### 4.3 Guardar un archivo .glt

Para guardar el codigo actual, ir al menu Archivo y seleccionar Guardar. Si el archivo ya tiene nombre, se guardara en la misma ubicacion. Si es un archivo nuevo, se abrira un dialogo para elegir la ubicacion y el nombre. El archivo se guardara con la extension .glt.

### 4.4 Ejecutar el codigo

Para interpretar el codigo escrito en el editor, ir al menu Ejecutar y seleccionar Ejecutar, o presionar el boton correspondiente en la interfaz. El interprete realizara el analisis lexico, sintactico y semantico, y ejecutara las instrucciones del programa.

La salida producida por las instrucciones `fmt.Println` aparecera en la consola de salida. Si se detectan errores en cualquier fase del analisis, estos se registraran en el reporte de errores.



The screenshot displays the GoLite IDE interface. The title bar reads "GoLite IDE - Nuevo archivo". The menu bar includes "Archivo", "Ejecutar", and "Reportes". The "Nuevo archivo" menu is open, showing a list of files with line numbers 226 through 241. The code in the editor is as follows:

```
func main() {  
226  
227 // =====  
228 // 1. IF ANIDADOS - 5 niveles  
229 // =====  
230 fmt.Println("--- 1. IF x5 ---")  
231  
232 a := 2  
233 b := 4  
234 c := 6  
235 d := 8  
236 e := 10  
237  
238 if a < b {  
239     if b < c {  
240         if c < d {  
241             if d < e {  
                if e < 10 {  
                    // ...  
                }  
            }  
        }  
    }  
}
```

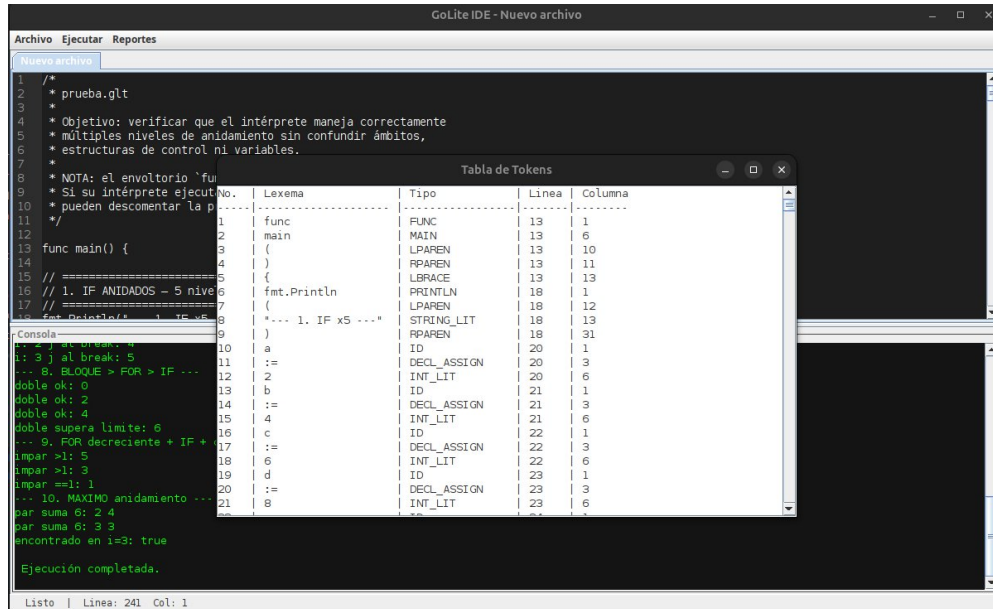
The console output at the bottom shows the execution results:

```
i: 2 j al break: 4  
i: 3 j al break: 5  
--- 8. BLOQUE > FOR > IF ---  
doble ok: 0  
doble ok: 2  
doble ok: 4  
doble supera limite: 6  
--- 9. FOR decreciente + IF + continue ---  
impar >1: 5  
impar >1: 3  
impar ==1: 1  
--- 10. MAXIMO anidamiento ---  
par suma 6: 2 4  
par suma 6: 3 3  
encontrado en i=3: true  
Ejecución completada.
```

The status bar at the bottom indicates "Listo | Linea: 241 Col: 1".

## 4.5 Ver el reporte de tokens

Después de ejecutar el código, se puede consultar la tabla de tokens reconocidos durante el análisis léxico. Esta tabla muestra el número de token, el lexema original, el tipo de token, la línea y la columna donde fue encontrado.

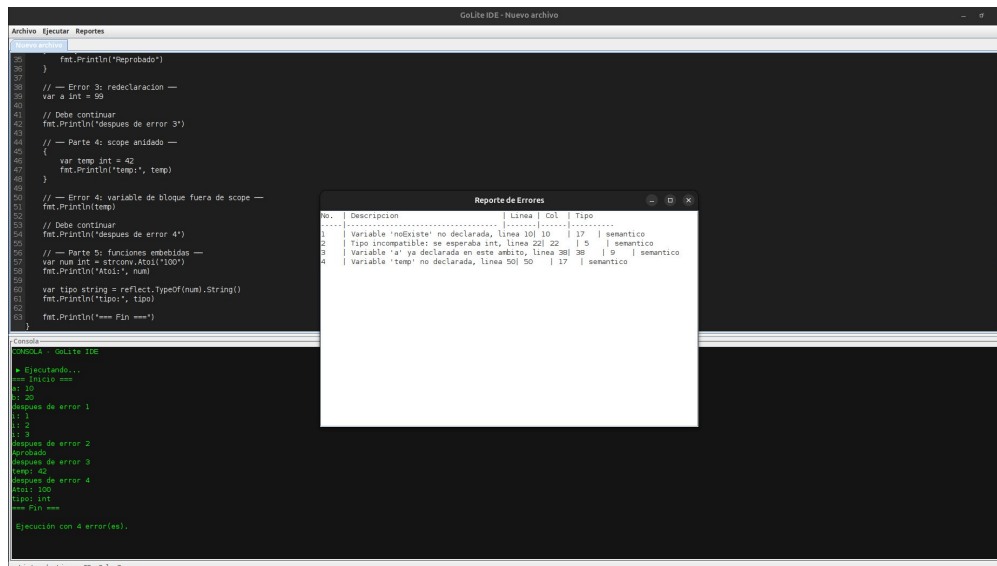


The screenshot shows the GoLite IDE interface. The main editor contains a C program named 'prueba.clt'. A 'Tabla de Tokens' window is open, displaying a table of tokens. The console shows the output of the program's execution.

| No. | Lexema             | Tipo        | Línea | Columna |
|-----|--------------------|-------------|-------|---------|
| 1   | func               | FUNC        | 13    | 1       |
| 2   | main               | MAIN        | 13    | 6       |
| 3   | (                  | LPAREN      | 13    | 10      |
| 4   | )                  | RPAREN      | 13    | 11      |
| 5   | {                  | LBACE       | 13    | 13      |
| 6   | fmt.Println        | PRINTLN     | 18    | 1       |
| 7   | (                  | LPAREN      | 18    | 12      |
| 8   | "... 1. IF x5 ..." | STRING_LIT  | 18    | 13      |
| 9   | )                  | RPAREN      | 18    | 31      |
| 10  | a                  | ID          | 20    | 1       |
| 11  | :=                 | DECL_ASSIGN | 20    | 3       |
| 12  | 2                  | INT_LIT     | 20    | 6       |
| 13  | b                  | ID          | 21    | 1       |
| 14  | :=                 | DECL_ASSIGN | 21    | 3       |
| 15  | 4                  | INT_LIT     | 21    | 6       |
| 16  | c                  | ID          | 22    | 1       |
| 17  | :=                 | DECL_ASSIGN | 22    | 3       |
| 18  | 6                  | INT_LIT     | 22    | 6       |
| 19  | d                  | ID          | 23    | 1       |
| 20  | :=                 | DECL_ASSIGN | 23    | 3       |
| 21  | 8                  | INT_LIT     | 23    | 6       |

## 4.6 Ver el reporte de errores

Si durante el análisis se detectan errores léxicos, sintácticos o semánticos, estos se muestran en el reporte de errores. Cada error indica su número, una descripción del problema, la línea, la columna y el tipo de error (léxico, sintáctico o semántico).



The screenshot shows the GoLite IDE interface. The main editor contains C code with several errors. A 'Reporte de Errores' window is open, displaying a table of errors. The console shows the output of the program's execution, including error messages.

| No. | Descripción                                             | Línea | Col. | Tipo      |
|-----|---------------------------------------------------------|-------|------|-----------|
| 1   | Variable 'redigster' no declarada, línea 10   17        | 10    | 17   | semántico |
| 2   | Tipo incompatible: se esperaba int, línea 22   22       | 22    | 22   | semántico |
| 3   | Variable 'a' ya declarada en este ámbito, línea 30   30 | 30    | 30   | semántico |
| 4   | Variable 'temp' no declarada, línea 50   50             | 50    | 50   | semántico |

## 5. Referencia del Lenguaje GoLite

Esta seccion describe brevemente las construcciones del lenguaje GoLite que el interprete puede ejecutar en la Fase 1.

### 5.1 Tipos de datos

| Tipo    | Descripcion                    | Valor por defecto | Ejemplo               |
|---------|--------------------------------|-------------------|-----------------------|
| int     | Numero entero                  | 0                 | var x int = 10        |
| float64 | Numero decimal de 64 bits      | 0.0               | var y float64 = 3.14  |
| string  | Cadena de caracteres           | ""                | var s string = "hola" |
| bool    | Valor logico verdadero o falso | false             | var b bool = true     |
| rune    | Caracter Unicode (alias uint8) | 0                 | var c rune = 'A'      |

### 5.2 Declaracion de variables

Se puede declarar una variable de tres formas:

```
var nombre int = 10           // tipo explicito con valor
var nombre int                // tipo explicito, valor por defecto
nombre := 10                  // inferencia de tipo
```

### 5.3 Operadores disponibles

| Categoria               | Operadores      | Ejemplo                       |
|-------------------------|-----------------|-------------------------------|
| Aritmeticos             | + - * / %       | 10 + 5 * 2 -> 20              |
| Comparacion             | == != < > <= >= | 10 > 5 -> true                |
| Logicos                 | &&    !         | true && false -> false        |
| Asignacion compuesta    | += -=           | x += 5 (equivale a x = x + 5) |
| Incremento / decremento | ++ --           | i++ (equivale a i += 1)       |

### 5.4 Sentencia if-else

```
if condicion {
    // instrucciones
} else if otraCondicion {
    // instrucciones
```

```

    } else {
        // instrucciones
    }

```

## 5.5 Sentencia for

Forma tipo while (ejecuta mientras la condicion sea verdadera):

```

for condicion {
}

```

Forma clasica con inicializacion, condicion e incremento:

```

for i := 0; i < 10; i++ {
}

```

## 5.6 Funciones embebidas

| Funcion                    | Descripcion                                                                 | Ejemplo                         |
|----------------------------|-----------------------------------------------------------------------------|---------------------------------|
| fmt.Println(...)           | Imprime uno o mas valores separados por espacio con salto de linea al final | fmt.Println("Hola", 42)         |
| strconv.Atoi(s)            | Convierte una cadena a entero. Error si no es un numero valido.             | n := strconv.Atoi("123")        |
| strconv.ParseFloat(s)      | Convierte una cadena a float64. Error si no es un numero valido.            | f := strconv.ParseFloat("3.14") |
| reflect.TypeOf(x).String() | Retorna el tipo de la expresion como cadena.                                | reflect.TypeOf(10).String()     |

## 6. Ejemplo Completo de Uso

A continuacion se muestra un ejemplo de sesion completa usando el interprete.

### Paso 1: Abrir el programa

Ejecutar el JAR como se indico en la seccion 2.2. La ventana principal aparecera en pantalla.

### Paso 2: Escribir el codigo

En el editor, escribir el siguiente programa de ejemplo:

```

func main() {

```

```

    var nombre string = "GoLite"
    var version int = 1
    fmt.Println("Interprete:", nombre, "Version:", version)
    for i := 1; i <= 5; i++ {
        fmt.Println("Iteracion:", i)
    }
}

```

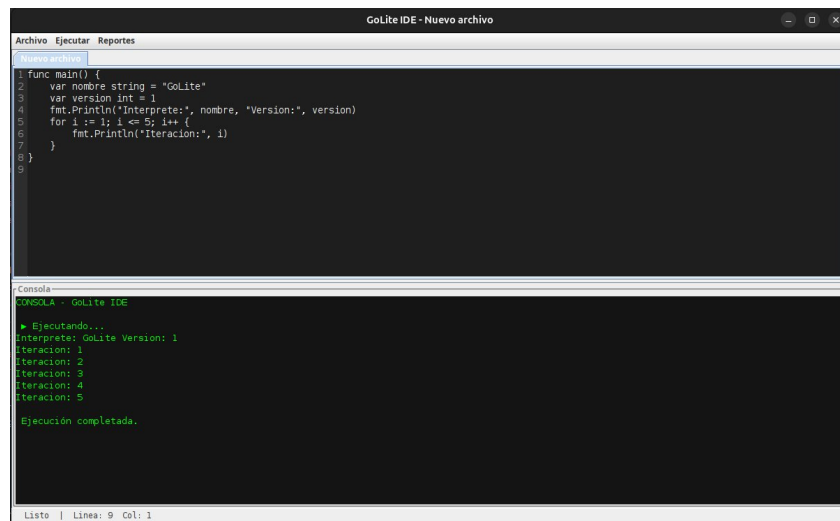
### Paso 3: Ejecutar

Ir al menu Ejecutar y seleccionar Ejecutar. La consola de salida mostrara:

```

Interprete: GoLite Version: 1
Iteracion: 1
Iteracion: 2
Iteracion: 3
Iteracion: 4
Iteracion: 5

```



### Paso 4: Revisar la tabla de tokens

Abrir el reporte de tokens para ver todos los tokens reconocidos en el código anterior.

### Paso 5: Guardar el archivo

Ir al menu Archivo y seleccionar Guardar. Asignar el nombre deseado al archivo y guardarlo con extensión .glt.



## 7. Mensajes de Error y Solucion

| Tipo       | Descripcion del error                                  | Causa                                                | Solucion                                                        |
|------------|--------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------|
| Lexico     | Caracter no reconocido: '@'                            | Se uso un caracter que no pertenece al lenguaje.     | Revisar el codigo y eliminar el caracter no valido.             |
| Sintactico | Error sintactico cerca de 'else'                       | El bloque if no tiene llaves o le falta algun token. | Verificar que todos los bloques esten bien delimitados con { }. |
| Semantico  | Variable 'x' no declarada en este ambito               | Se uso una variable antes de declararla.             | Declarar la variable antes de usarla con var o :=.              |
| Semantico  | No se puede asignar float64 a una variable de tipo int | Se intento asignar un valor de tipo incompatible.    | Usar el tipo correcto o declarar la variable como float64.      |
| Semantico  | Division entre cero                                    | El divisor de una operacion es cero.                 | Verificar que el denominador no sea cero antes de dividir.      |
| Semantico  | break fuera de un ciclo                                | Se uso break o continue fuera de un for.             | Usar break y continue unicamente dentro de un ciclo for.        |

## 8. Archivos Soportados

- El editor abre y guarda archivos con extension .glt (GoLite source file).
- El archivo debe estar codificado en UTF-8.
- No hay limite de tamano de archivo impuesto por el interprete, pero archivos muy grandes pueden tardar mas tiempo en analizarse.

## 9. Notas Adicionales

- El lenguaje GoLite es sensible a mayusculas y minusculas (case-sensitive). La palabra if no es lo mismo que IF.
- Las llaves { } son obligatorias en todas las estructuras de control. No se admite la forma de una sola linea sin llaves.
- El punto y coma al final de cada sentencia es opcional.
- Los comentarios de una sola linea se escriben con // y los comentarios de varias lineas con /\* ... \*/.
- Una variable declarada en un bloque interno oculta a una variable con el mismo nombre en un bloque externo, y deja de existir al terminar ese bloque.
- Los tipos no pueden cambiar una vez declarada la variable. Si se necesita otro tipo, se debe declarar una nueva variable.